

AR Foundation

Kursüberblick

1. Intro
2. Mobile UX & Input
3. Performance & Mobile Constraints
4. GPS & Location-Based Mechanics
5. Map Integration
-  6. AR Foundation
7. Game Loop & Systems
8. Polishing + Smartphone Features
9. Prüfung

Arten von AR

Marker Based AR

Beispiel:

- QR Code
- Bildtracking

Eigenschaften:

- einfach
- stabil
- braucht Referenzbild

Location-Based AR

Beispiel:

- GPS
- Compass
- Gyroscope

Die Map-Szene IST bereits eine Form von AR-Gameplay.

Aber:

- ungenau
- kein echtes Raumverständnis

Spatial AR

Beispiel:

- Plane Detection
- Feature Points
- Anchors

Eigenschaften:

- Das Gerät analysiert Kamerabilder und Sensoren, erkennt markante Punkte und rekonstruiert daraus die Umgebung.

Mobiles AR

Sensoren

Smartphone nutzt:

- Kamera
- Gyroscope
- Accelerometer
- manchmal Depth Sensor / LiDAR

Plane Detection

Das System sucht:

- Böden
- Tische
- Wände

und approximiert diese als Ebenen.

Wichtig:

- gute Lichtverhältnisse
- strukturierte Oberflächen
- Bewegung nötig



Anchors

SEHR wichtig

- Ein Anchor ist ein stabiler Referenzpunkt in der echten Welt.

Ohne Anchor:

- Objekte driften
- springen
- zittern

Mit Anchor:

- Objekt bleibt stabil „an Ort und Stelle“

Unity AR Foundation

Unity AR Foundation

AR Foundation ist das Unity Abstraktionslayer

Darunter:

- ARKit (iOS)
- ARCore (Android)
- Oculus
- OpenXR

Unity AR Foundation - Installieren

1. Package Manager -> Sources -> Unity Registry -> AR Foundation
2. In Project Settings -> XR Plug-in Management
 - a. -> Google ARCore aktivieren
3. Neuen Renderer erstellen URP Universal Renderer und im **UniversalRP** verknüpfen
4. Auf neuem Renderer-> Add Render Feature -> **ARCommandBufferSupportRendererFeature** & **ARBackgroundRendererFeature**

AR Scene

1. Neue Szene
2. Create -> AR Session
3. Create -> XROrigin
 - a. Kamerawerte einstellen
4. API Level auf 29 hochsetzen!
5. Ein Monster einfügen
6. Build & Test

Kurze Pause?

AR in Unity

Unity AR - Komponenten

- AR Session
 - Verwaltet AR Lifecycle.
- AR Session Origin / XR Origin
 - Koordinatensystem, trackt alle XR Objekte
- AR Camera
 - Tracked Device Camera.
- AR Plane Manager
 - Erkennt Ebenen.
- AR Raycast Manager
 - Screen → reale Welt.
- AR Anchor Manager
 - Stabile Platzierung von virtuellen Objekten

Unity AR - Planes erkennen

1. auf **XROrigin** Komponente **ARPlaneManager** hinzufügen
 - a. Tracking Mode auf **Horizontal** stellen
2. Create GameObject -> AR Default Plane
 - a. Umbenennen
 - b. Prefabisieren
 - c. Prefab aus Szene löschen
 - d. in ARPlaneManager verknüpfen
3. **Testen!**

Creatures Spawned

Unity AR - Creature Spawnen!

Ziel für diese Szene:

1. Planes erkennen
2. “Beste” Plane auswählen
 - a. “beste” Plane ist die größte!
3. Creature dort Spawnen
 - a. auf Scaling achten!
 - b. Unity Einheiten sind 1= 1m!

Unity AR - Creature Spawnen

1. Plane erkennen: Macht der Plane Manager automatisch
2. Nach dem Starten der Szene mindestens ein paar Sekunden scannen lassen,
 - a. Erst dann nach der “besten” Plane suchen
3. Auf dieser dann die Creature platzieren
4. Anschließend alle “Flächenvisuals” ausmachen

```
// Wir gehen durch alle aktuell erkannten Flächen durch
foreach (var plane in planeManager.trackables)
{
    // Die Größe der Fläche berechnen (Breite * Länge des Extents)
    float planeArea = plane.extents.x * plane.extents.y;
```

```
// Spawn Position ist .center der ARPlane
Vector3 spawnPosition = bestPlane.center;
```

Optionaler Hinweis Text: “Creature spawn! Bewege die Kamera leicht!”

-> beim Spawnen dann ausmachen

Unity AR - Creature Spawnen - Tipp

Damit die Creature immer zum Spieler schaut:

```
// Berechnen die Richtung zum Spieler (Kamera)
Vector3 directionToCamera = Camera.main.transform.position - spawnPosition;

// Wir nullen die Y-Achse, damit das Monster nicht nach oben/unten kippt, falls der Tisch tiefer liegt
directionToCamera.y = 0;

// Erzeugt eine Rotation, die exakt in diese Richtung blickt
Quaternion spawnRotation = Quaternion.LookRotation(directionToCamera);
```

Object Drift

Die Creature ist gespawnt. Aber sie ist nur an das Unity Koordinatensystem geknüpft.

Das ist starr. Aber die Kamera scannt weiter und korrigiert heimlich die Welt.

-> Das Objekt “driftet” visuell herum.

Die Lösung: **Anchors!**

Dabei wird zusätzlich zur Kameraposition selber auch dieser Punkt im Raum exakt getrackt. -> Ergebnis: Die Creature bleibt fest!

Anchors

1. Auf **XROrigin**
 - a. Neue Komponente **ARAnchorManager**
2. CreatureSpawner erweitern:
 - a.

```
spawnedCreature.AddComponent<ARAnchor>();
```
 - b. Dadurch fügen wir einen Anchor hinzu, Unity übernimmt die Registrierung im Anchor Manager
3. Im ARAnchorManager noch ein "Debug" Prefab hinzufügen
 - a. z.B: rote Sphere mit (0.1,0.1,0.1) Scale
4. Build & Test
5. Creature spawn und hat "Anchor" -> kein Sliding mehr
6. Anschließend "Debug Anchor" wieder rausnehmen

Creature ködern

Unity AR - Creature ködern!

Ziel für diese Szene:

1. Köder vor der Creature auslegen
 - a. Auf Screen tippen
 - b. Raycast in die "Welt"
 - c. dort dann Köder spawnen

Dafür nutzen wir den **ARRaycastManager**!

Food Assets:

<https://github.com/Tillomaticus/MobileGameProg26Assets/blob/6-AR/Kenney%20-%20Food%20Pack.unitypackage>

Unity AR - Creature ködern!

1. Auf **XROrigin** den **ARRaycastManager** hinzufügen
2. Neues Script schreiben: ARBaitSpawner
 - a. raycastManager (verknüpfen)
 - b. playerInputSystem **OnPrimaryTouch** nutzen -> aus unserem alten Input Script kopieren
3. OnPrimaryTouch:
 - a. `raycastManager.Raycast` von Touch-Position,
 - b. hits: `List<ARRaycastHit> hits`
 - c. `TrackableType.PlaneWithinPolygon`
 - d. Wenn hit, dann an hit position baitPrefab spawnen!
 - e. *(optional) alten Bait despawnen und durch neuen ersetzen*

Unity AR - Creature ködern! Lösung

```
// Der AR-Raycast: Wir schießen einen Strahl von der Touch-Position auf dem Display
// in die echte Welt. Wir wollen NUR echte, erkannte Flächen treffen (PlaneWithinPolygon).
if (raycastManager.Raycast(_playerInput.Touch.PrimaryTouchPosition.ReadValue<Vector2>(),
hits, TrackableType.PlaneWithinPolygon))
{
    // hits[0] ist immer der Treffer, der der Kamera am nächsten ist
    Pose hitPose = hits[0].pose;
```

```
if (spawnedBait == null)
{
    // Köder existiert noch nicht? Neu spawnen!
    spawnedBait = Instantiate(baitPrefab, hitPose.position, hitPose.rotation);
}
else
{
    // Optional: Wenn schon ein Köder da ist, versetzen wir ihn einfach
    Destroy(spawnedBait);
    spawnedBait = Instantiate(baitPrefab, hitPose.position, hitPose.rotation);
}
```

Testen &
Kurze Pause?

Creature einfangen

Aufgabe - Creature fangen!

1. Creature sieht Bait
 - a. z.B. über Action in BaitSpawner
2. Creature bewegt sich auf Bait zu
3. Sobald Creature nahgenug an Bait ist
 - a. Creature gefangen
4. Creature gefangen
 - a. Creature verstecken
 - b. Bait werfen deaktivieren
 - c. UI anzeigen mit "Gefangen!"
 - d. Button um zurück zur "Hauptszene" zu kommen

Lösung - Creature fangen!

ARBaitSpawner erweitern:

```
public static event Action<Transform> OnBaitPlaced;

// Event auslösen
OnBaitPlaced?.Invoke(spawnedBait.transform);
```

CreatureLogic

```
void OnEnable()
{
    // Auf das statische Event vom BaitSpawner subscriben
    ARBaitSpawner.OnBaitPlaced += HandleNewBait;
}

// Wird aufgerufen, sobald das Event im Spawner feuert
2 references
private void HandleNewBait(Transform baitTransform)
{
    targetBait = baitTransform;
    isMoving = true;
}
```

```
void Update()
{
    // Wenn wir kein Ziel haben oder nicht laufen sollen, tun wir nichts
    if (!isMoving || targetBait == null) return;

    // 1. Blickrichtung zum Köder drehen (sieht natürlicher aus)
    Vector3 targetPosition = new Vector3(targetBait.position.x,
    transform.position.y,
    targetBait.position.z);

    transform.LookAt(targetPosition);

    // 2. Auf den Köder zubewegen
    transform.position = Vector3.MoveTowards(transform.position, targetPosition, speed * Time.deltaTime);

    // 3. Distanz prüfen: Sind wir nah genug dran?
    if (Vector3.Distance(transform.position, targetPosition) <= stopDistance)
    {
        isMoving = false; // Bewegung stoppen
        StartCoroutine(ARSceneManager.Instance.TriggerCatchUIRoutine()); // UI-Verzögerung starten
        this.gameObject.SetActive(false);
    }
}
```


Verbessern der Szene

1. Bait erst nutzbar wenn Creature gespawnt ist
 - a. (optional) verschiedene Bait Typen
2. Creature Animation
 - a. Idle Animation
 - b. Walk Animation
 - c. "Fress" Animation
3. Fürs "Fangen" Creature beim Fressen antippen